Ui Dev Guide

Volume 1

50+ es Interview Questions

Ultimate Guide for 2024

ES6

Welcome to the ES6 Interview Guide!

This eBook is designed to help you master the essential concepts of ES6 and ace your JavaScript interviews. Here's a quick guide on how you should read and practice the material to get the most out of it.
Follow the steps to upscale for your next interview:

- **Understand the Basics First**
- **Practice Coding After Each Topic**
- **Use Interactive Resources**
- **Apply the Concepts in Small Projects**
- **Use Online Coding Platforms for Practice**
- **Schedule mock Review Sessions**
- **Ask Questions and Join Communities**
- **Prepare for Interviews by Mock Testing**

Frequently Asked ES6 Interview Questions

What is ES6?

ES6, or ECMAScript 2015, is the sixth version of JavaScript, marking a significant upgrade since ES5 in 2009. It introduced several new features such as classes, modules, and advanced object manipulation techniques.

How many parts are in an ES6 module?

While modules have long been a part of the JavaScript ecosystem through libraries, they became native to the language in ES6. Modules are reusable blocks of code, which consist of two main parts: Default Export and Named Export. You can import these modules in other files for reuse.

What is a class in ES6?

Introduced in 2015 with ES6, classes in JavaScript provide a blueprint for creating objects, encapsulating data, and defining methods. Example:

```
Class Order {
  constructor(id) {
    this.id = id;
  }

  getId() {
    console.log('The Order Id of Your Order ${this.id}')
  }
}

Order o1 = new Order('5gf123');
o1.getId(); Output: The Order Id of Your Order 5 gf123
```

What is an arrow function in ES6?

Arrow functions serve as a shorter syntax for function expressions, with lexical scoping for the `this` keyword. Unlike traditional functions, arrow functions cannot be used as methods or constructors. Example:

```
const greet = () => {
  console.log("Hello Guys!");
};
```

What is `export default` in ES6?

The `export` statement is used to make code available to other modules. There are two types: Named Export and Default Export, the latter being used to export a single entity such as a variable, function, or object. Example:

```
// In Test.js
const x1 = 10;
export default x1;

// In Com.js
import y1 from './Test.js';
```

What is destructuring in ES6?

Destructuring allows extracting values from arrays or objects into distinct variables. Example:

```
let [dog, cat, rabbit] = ['dog', 'cat', 'rabbit'];
console.log(dog); // Output: dog
```

What is event propagation (capturing and bubbling)?

When an event is triggered in the DOM, it doesn't occur on just one element. It propagates through three phases:

- **Capturing Phase: The event moves from the window down to the target element.**
- **Target Phase: The event reaches the target element.**

- Bubbling Phase: The event bubbles back up to the window.

What is the difference between ES5 and ES6?

ES6, introduced in 2015, brought many new features compared to ES5 (2009):
Variables ES6 introduced `let` and `const` for block scoping, whereas ES5 had only `var` for function scoping.

- Destructuring: Simplifies the assignment of array and object values.
- Arrow Functions: Provide a shorter syntax and lexical scoping for `this`.
- Classes and Modules: Native support in ES6 for OOP and modularization.

How do you define a function in ES6?

ES6 introduced arrow functions. Here's a breakdown:
Remove the `function` keyword and place an arrow `=>` between the arguments and the body.
If the function body has only one expression, remove the body brackets and the `return` keyword is implied.

Parentheses around arguments are optional if there's only one argument.

```
const add = (a, b) => a + 100;
```

How do you import CSS in ES6?

In ES6, you can import CSS files directly into your JavaScript files using the `import` statement.
Example:

```
import './index.css';
```

What are the new features introduced in ES6?

Major features introduced in ES6 include:

- let and const: Block-scoped variables.
- Destructuring: Easily extract values from arrays and objects.
- Template Literals: Simplified string concatenation using backticks.
- Classes: Object-oriented syntax for creating objects.
- Modules: Native support for exporting and importing code.

Define `let` and `const` in ES6.

let: Allows block-scoped variables, preventing redeclaration in the same scope. However, it allows reassignment.

const: Creates block-scoped variables that cannot be reassigned or redeclared. Once a value is assigned to a `const` variable, it cannot be changed.

```
let x = 10;
x = 20; // This works

const y = 10;
y = 20; // This throws an error
```

What is Functional Programming?

Functional programming is a programming paradigm where functions are treated as first-class citizens. It emphasizes immutability, pure functions (no side effects), and higher-order functions like `map`, `filter`, and `reduce`. Example:

```
const numbers = [1, 2, 3, 4];
const doubled = numbers.map(n => n * 2)
```

Give an example of an Arrow function in ES6. and list its advantages.

Arrow functions provide a concise syntax for writing functions. They also lexically bind the `this` keyword, making them more predictable in certain contexts. Example:

```
const greet = name => `Hello, ${name}`;
```

Advantages of Arrow Functions:

- **Shorter syntax**
- **Lexical scoping of `this`**
- **No need for the `function` keyword**
- **Implicit return for single expressions**

What is the Spread Operator in ES6?

The spread operator (`...`) expands iterable elements like arrays or strings into individual elements. Example:

```
const arr1 = [1, 2, 3];
const arr2 = [...arr1, 4, 5];  // [1, 2, 3, 4, 5]
```

It can also be used to copy objects or merge arrays.

What are Template Literals in ES6?

Template literals allow easier string interpolation and multi-line strings using backticks (`` ` ``). Example:

```
const name = 'John';
const greeting = `Hello, ${name}!`;
```

This simplifies concatenation and supports embedded expressions.

What is a Generator Function?

A generator function is a special kind of function that can pause its execution (`yield`) and later resume from where it left off. Example:

```
function* gen() {
  yield 1;
  yield 2;
  yield 3;
}

const g = gen();
console.log(g.next().value); // Output: 1
```

Why are functions called first-class objects in JavaScript?

Functions in JavaScript are treated as first-class objects, meaning they can:

- Be assigned to variables.
- Be passed as arguments to other functions.
- Be returned from functions.
- Have properties and methods.

What is the Rest Parameter in ES6?

The rest parameter (`...`) allows a function to accept an indefinite number of arguments as an array. Example:

```
function sum(...numbers) {
  return numbers.reduce((total, num) => total + num, 0);
}

console.log(sum(1, 2, 3, 4)); // Output: 10
```

What do you mean by IIFE (Immediately Invoked Function Expression)?

An IIFE is a JavaScript function that is executed immediately after it is defined. It is wrapped in

parentheses to create its own scope and prevent polluting the global scope. Example:

```
(function() {
  console.log('This is an IIFE');
})();
```

What are default parameters in ES6?

ES6 introduced default parameters, allowing function arguments to have default values if no value or `undefined` is passed. Example:

```
function greet(name = 'Guest') {
  console.log(`Hello, ${name}`);
}

greet(); // Output: Hello, Guest
```

Explain the `for...of` loop in ES6?

The `for...of` loop iterates over iterable objects like arrays, strings, or maps. It provides an easy way to loop through the values. Example:

```
const animals = ['cat', 'dog', 'bird'];
for (const animal of animals) {
  console.log(animal);
}
// Output: cat, dog, bird
```

Explain the `for...in` loop.

The `for...in` loop iterates over the enumerable properties of an object (keys or properties). It is useful for working with objects rather than arrays. Example:

```
const employee = {
      name: 'John',
      age: 30,
      position: 'Developer'
    };

    for (let key in employee) {
      console.log(`${key}: ${employee[key]}`);
    }
```

What is a `Map` in ES6?

A `Map` is a collection of key-value pairs, where both keys and values can be of any data type. It provides better performance when dealing with a large number of elements. Example:

```
const map = new Map();
map.set('name', 'John');
console.log(map.get('name')); // Output: John
```

What is a `Set` in ES6?

A `Set` is a collection that holds unique values. It automatically eliminates duplicate entries. Example:

```
const set = new Set([1, 2, 3, 3, 4]);
console.log(set);  // Output: Set(4) {1, 2, 3, 4}
```

What is a `WeakMap` in ES6?

A `WeakMap` is similar to a `Map`, but its keys must be objects, and the keys are held weakly. This means that if there are no other references to a key object, it can be garbage collected. Example:

```
const weakMap = new WeakMap();
let obj = {};
weakMap.set(obj, 'value');
```

Explain promises in ES6.

A promise represents the eventual completion (or failure) of an asynchronous operation and its resulting value. A promise can be in one of three states: *pending*, *fulfilled*, or *rejected*. Example:

```
const promise = new Promise((resolve, reject) => {
  setTimeout(() => resolve('Success!'), 1000);
});

promise.then(result => console.log(result)); // Output: Success!
```

What is a `WeakSet` in ES6?

A `WeakSet` is a collection of objects where the objects are held weakly. This allows for garbage collection when there are no other references to the objects. Unlike `Set`, `WeakSet` can only hold objects, not primitive values. Example:

```
const weakSet = new WeakSet();
let obj = {};
weakSet.add(obj);
```

What is Callback and Callback Hell in JavaScript?

Callback: A function passed as an argument to another function, to be executed after the completion of a task.

Callback Hell: This occurs when multiple nested callbacks make code difficult to read and maintain. Example:

```javascript
setTimeout(() => {
  console.log('Task 1');
  setTimeout(() => {
    console.log('Task 2');
    setTimeout(() => {
      console.log('Task 3');
    }, 1000);
  }, 1000);
}, 1000);
```

What is hoisting in JavaScript?

Hoisting is JavaScript's default behavior of moving variable and function declarations to the top of the scope before code execution. Variables declared with `var` are hoisted, but only their declarations, not initializations.
`let` and `const` are hoisted but not initialized, so accessing them before declaration results in a `ReferenceError`.

```javascript
console.log(x);  // Output: undefined
var x = 5;
```

What is AJAX in JavaScript?

AJAX stands for Asynchronous JavaScript and XML. It enables web pages to be updated asynchronously by exchanging data with a web server behind the scenes. This allows web pages to update dynamically without needing to reload the entire page.

Technologies involved in AJAX include:

- **HTML (structure)**
- **CSS (styling)**
- **JavaScript (behavior)**
- **XMLHttpRequest (to interact with the server)**

List some new array methods introduced in ES6.

- **concat(): Merges two or more arrays.**
- **every(): Checks if all elements pass a test.**
- **filter(): Creates a new array with elements that pass a test.**
- **find(): Returns the value of the first element that satisfies a condition.**
- **forEach(): Executes a function for each array element.**
- **Array.from(): Converts array-like or iterable objects into arrays.**

What are the new string methods introduced in ES6?

- **startsWith():** Checks if a string starts with specified characters.
- **endsWith():** Checks if a string ends with specified characters.
- **includes():** Checks if a string contains specified characters.
- **repeat():** Repeats a string a given number of times.

What is Webpack?

Webpack is a module bundler for JavaScript. It compiles JavaScript modules into a single file or a set of files, making it easier to manage dependencies. Webpack is commonly used in front-end development but can also work with Node.js on the backend.

What is Babel?

Babel is a JavaScript transpiler that converts newer ECMAScript (like ES6) syntax into ES5 or earlier, making the code compatible with older browsers. It allows developers to use modern JavaScript features without worrying about browser support.

```
// Input (ES2020):
const greet = name => `Hello, ${name ?? "World"}`

// Output (ES5):
var greet = function(name) {
  return "Hello, " + (name != null ? name : "World");
};
```

What are the object-oriented features in ES6?

Some of the object-oriented features introduced in ES6 include:

- Classes: Introduces a simpler syntax for defining objects.
- Inheritance: Using the `extends` keyword.
- Static Methods: Methods that belong to the class itself, not instances.
- Getters and Setters: Methods that get and set object properties.

Compare ES5 and ES6.

- ES5 (2009): Supports `var`, function declarations, and objects.
- ES6 (2015): Adds `let`, `const`, classes, template literals, arrow functions, and modules.
- Variable Declaration: ES5 uses `var`, while ES6 introduces `let` and `const`.
- Arrow Functions: Not present in ES5 but introduced in ES6 for simpler syntax.
- Modules: Native module support in ES6.

What is the difference between `let`, `const`, and `var`?

- let: Block-scoped, can be updated but not redeclared.
- const: Block-scoped, cannot be updated or redeclared after initialization.
- var: Function-scoped, can be redeclared and updated, prone to hoisting issues.

Name some array methods introduced in ES6.

- Array.fill(): Fills an array with a static value.
- Array.from(): Converts array-like objects into arrays.
- Array.prototype.keys(): Returns an iterator containing the keys of the array.
- Array.of(): Creates an array from its arguments.

What are some string methods introduced in ES6?

- includes(): Checks if a string contains a specified value.
- startsWith(): Checks if a string starts with a specified value.
- endsWith(): Checks if a string ends with a specified value.
- repeat(): Returns a new string with a specified number of copies.

How do you use destructuring assignment to swap variables?

In ES6, destructuring can be used to swap variables in a concise manner:

```
let a = 1, b = 2;
[a, b] = [b, a];
console.log(a); // Output: 2
console.log(b); // Output: 1
```

What is the result of spreading the string `'macbook'` using the spread operator?

When you spread a string, it breaks it into individual characters:

```
console.log([...`macbook`]);
// Output: ['m', 'a', 'c', 'b', 'o', 'o', 'k']
```

What is the Prototype Design Pattern?

The prototype design pattern is used to create objects based on a template, or "prototype," from which other objects are cloned. In JavaScript, this is inherent since JavaScript is a prototype-based language where objects inherit properties and methods from other objects.

What is the difference between `WeakMap` and `Map` in ES6?

Map: Holds key-value pairs, where keys can be any data type (including primitives), and the key-value pairs are strongly referenced.

WeakMap: Only accepts objects as keys, and the keys are weakly referenced, meaning they can be garbage collected if no other references exist.

What is the advantage of using arrow functions for constructor methods?

Arrow functions do not have their own `this`, so they inherit the `this` value from their surrounding context. This makes them more predictable inside constructors, as they always reference the class instance.

What is the Temporal Dead Zone in JavaScript?

The Temporal Dead Zone (TDZ) is the time between the declaration and initialization of a `let` or `const` variable. Accessing the variable within this zone results in a `ReferenceError`.

```
console.log(a); // Throws ReferenceError
let a = 10;
```

What is the difference between `Set` and `WeakSet` in ES6?

- **Set: A collection of unique values (both primitives and objects). Values in `Set` are strongly held.**
- **WeakSet: Only holds objects, and these objects are weakly referenced, allowing garbage collection when no other references exist.**

What is a Proxy in ES6?

A Proxy object in ES6 allows developers to define custom behavior for fundamental operations on objects, such as reading and writing properties, function calls, and more. Proxies can be used for tasks like validation or logging.

What is the difference between `const` and `Object.freeze()`?

const: Prevents reassignment of a variable.
Object.freeze(): Prevents modification of an object's properties (deep freeze must be used for nested objects).

Why does the following not work as an IIFE (Immediately Invoked Function Expression)? What needs to be modified?

```
function x() {

}();
```

The above code doesn't work as an IIFE because the JavaScript engine interprets the `function x() {}` as a function declaration, and function declarations cannot be immediately invoked. To fix this, you can wrap the function in parentheses:

```
(function x() {})(); // or (function() {})();
```

What is Internationalization and Localization in JavaScript?

Internationalization (i18n): It is the process of designing a product so that it can be adapted to various languages and regions without requiring engineering changes.

Localization (l10n): It is the process of adapting a product to a specific locale or market, such as translating text into a different language or formatting dates and numbers.

List the advantages of Arrow Functions.

- Concise syntax, which reduces code size.
- No need for the `return` keyword in single-line functions.
- Implicit binding of `this`, providing a more predictable behavior.
- No need for curly braces in single-expression functions.

Explain destructuring assignment in ES6.

Destructuring allows you to extract data from arrays or objects into distinct variables. Example:

```
const person = {
  name: 'John',
  age: 30
};
const {
  name,
  age
} = person;
console.log(name); // Output: John
```

What is a `Map` in ES6?

A `Map` is a collection of key-value pairs, where keys and values can be of any data type. Unlike objects, `Map` keys can be anything, including objects or functions. Example:

```
const map = new Map();
map.set('key1', 'value1');
map.set({}, 'value2');
console.log(map.get('key1')); // Output: value1
```

Explain the `for...of` loop with an example.

The `for...of` loop iterates over the values of iterable objects like arrays or strings. Example:

```
const colors = ['red', 'green', 'blue'];
for (const color of colors) {
  console.log(color);
}
```

What is a `WeakSet` in ES6?

A `WeakSet` is a collection of objects, where objects are held weakly. This means that if no other references to an object exist, it can be garbage collected. Example:

```
let obj1 = {
  name: 'John'
};
const weakSet = new WeakSet();
weakSet.add(obj1);
```

Mention the 3 states of Promises in ES6.

- **Pending: The initial state, before the promise is fulfilled or rejected.**
- **Fulfilled: The operation was completed successfully.**
- **Rejected: The operation failed.**

What are modules in JavaScript?

Modules allow you to encapsulate code into reusable components. In ES6, modules can be created using `export` and `import` statements. Example:

```
// module.js
export const name = 'John';

// main.js
import {  name } from './module.js';
console.log(name); // Output: John
```

Explain named export and default export in ES6.

Named Export: Exports multiple values, and the names must match during import.
Example:

```
export const x = 10;
export function greet() {
  console.log('Hello');
}
import {
  x,
  greet
} from './module.js';
```

Default Export: Allows exporting a single value or object. Import names can differ from export names.

Example:

```
export default function greet() {
  console.log('Hello');
}
import greetFunction from './module.js';
```

To enhance your learning, I invite you to check out my YouTube channel: UI Dev Guide. The channel is dedicated to providing mock interview for job seeker, helping in career consulting, and practical examples that mostly asked in the interviews
check below playlist for javascript angular and react:

Angular : 🌐 Must Watch Angular interviews
React: 🌐 Must watch React interviews list before going for interview
Javacript Coding :
🌐 JavaScript coding interview questions | JavaScript interview questions and answers
Javascript: 🌐 JavaScript interview questions